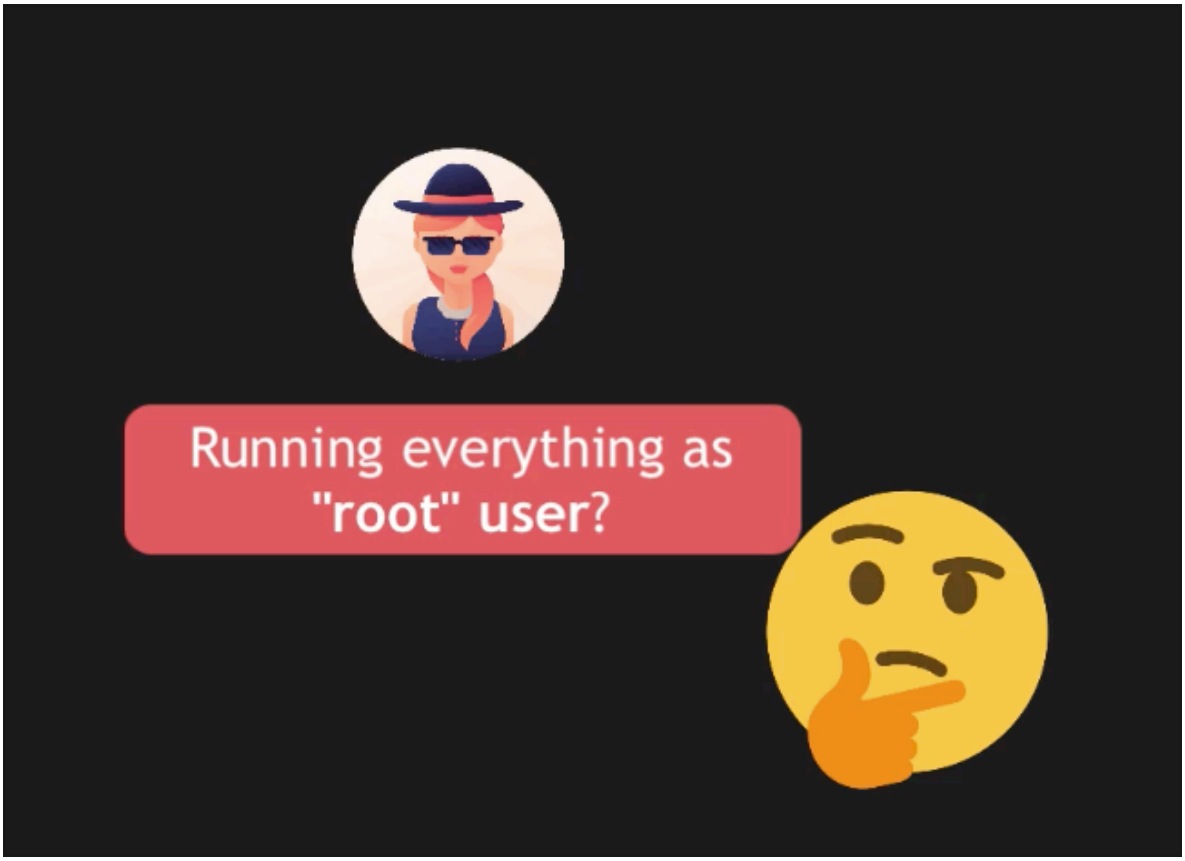
Module 15 - chapter 11

Project: Deploy Nodejs application - Part 3

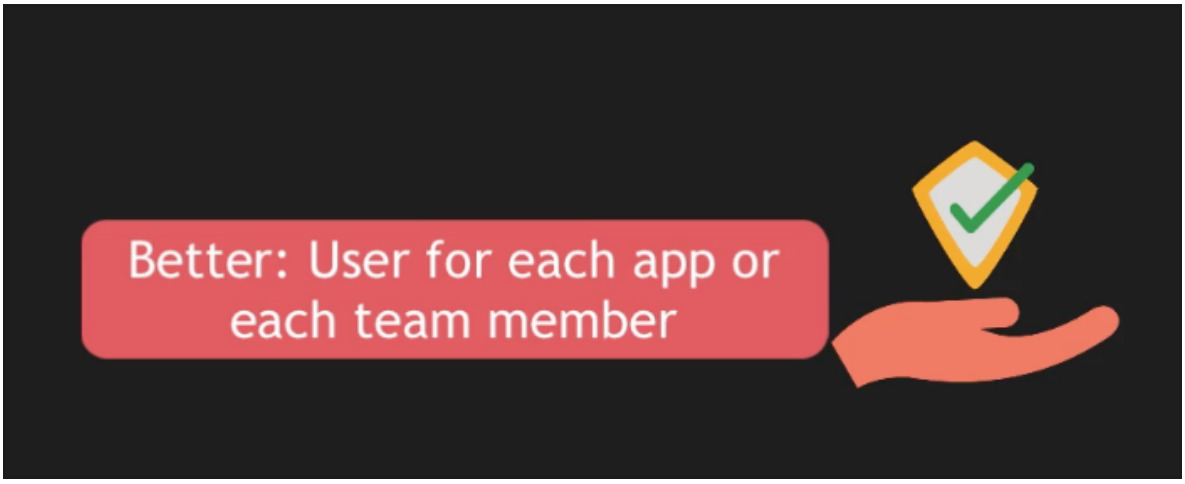
The final YAML file used in this lecture can be found here: https://gitlab.com/twn-devops-bootcamp/latest/15-ansible/ansible-projects/-/blob/main/deploy-node.yaml

Executing tasks with a different user

We have a playbook that can now be executed in any remote server and we have use the 'root' user ‼️

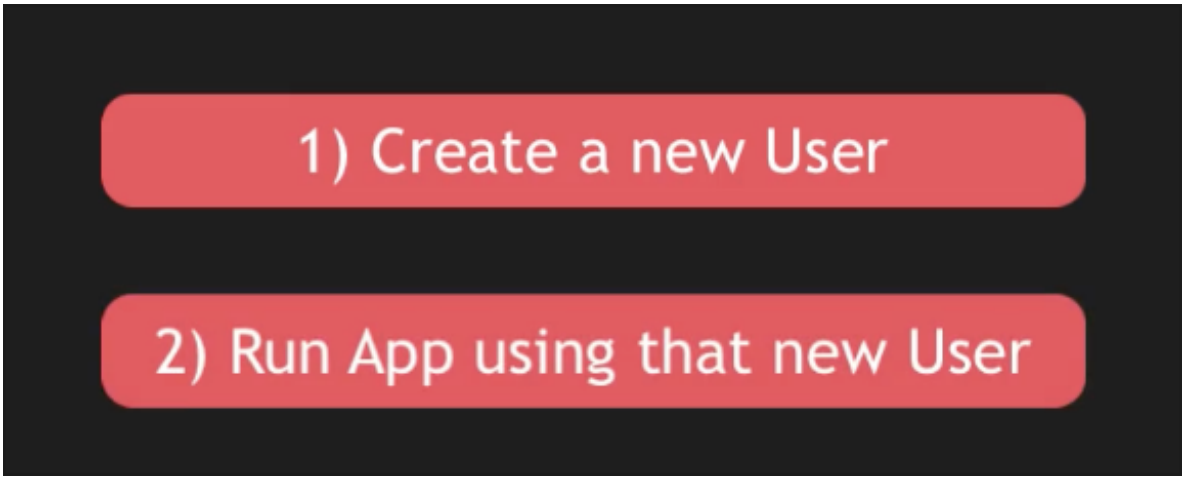


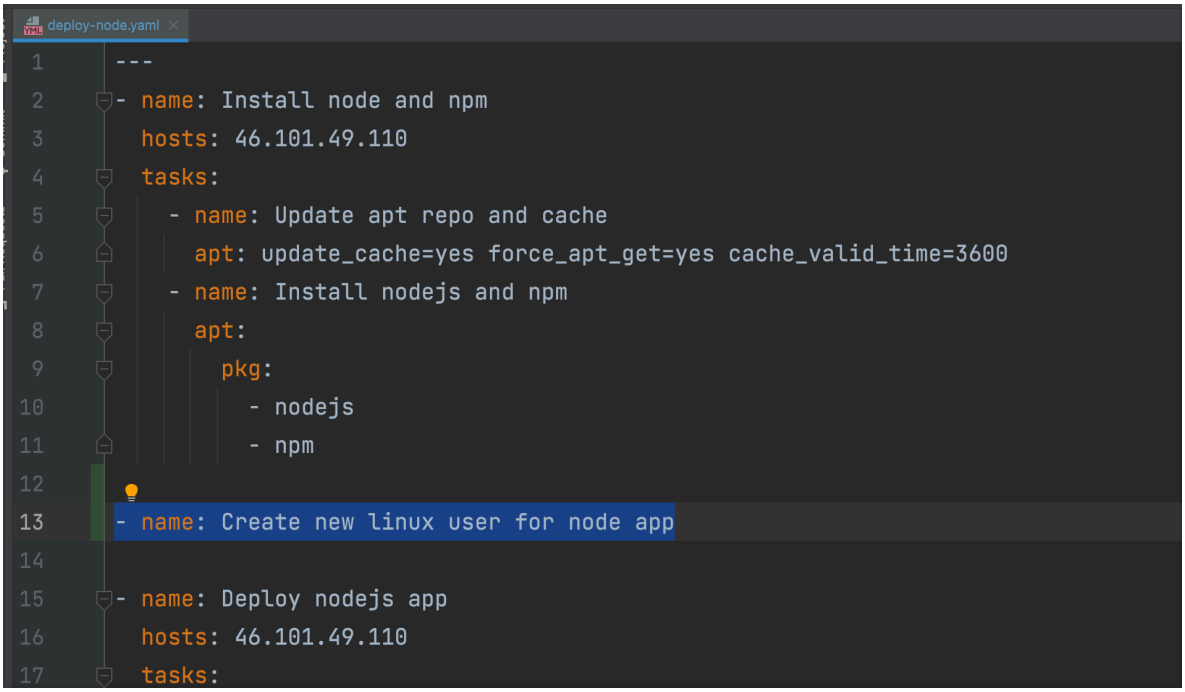
And we know this is not best practice



And we should start the app with a non-root user.

So we need to create a new user in the remote server as at the moment our droplet only has one user which is ‘root’





We keep the first play as it is because it needs root privileges.

And we create the new play to create the new user using the ‘user’ module:

And we add the user to a group and we will call that group ‘admin’ which has a lots of privileges but not as much as ‘root’.

Q:
the droplet ubuntu comes with admin group set up?

A:
Yes, that's right — but it's specific to your droplet's image, not vanilla Ubuntu itself. Standard/official Ubuntu images (from Canonical) configure the sudo group for admin rights, not admin.

DigitalOcean's own Ubuntu droplet images, however, have historically kept this

legacy admin group + the %admin ALL=(ALL) ALL sudoers line baked in (a holdover from pre-12.04 Ubuntu conventions), in addition to sudo.

So: it's a DigitalOcean image customization, not something guaranteed on every Ubuntu install (e.g. a bare-metal Ubuntu server or a different cloud provider's Ubuntu image might only have sudo). Good to keep in mind if you ever move this playbook to a non-DigitalOcean host — you'd want to check for admin again rather than assume it's there.

I checked if the droplet has the group admin
-> getent group admin

```
root@nodejs-app:~# getent group admin
admin:x:110:
```

Then I checked if the admin group is wired up to give sudo rights. One thing worth checking though: on Ubuntu, the group that actually grants sudo/admin privileges is normally sudo, not admin. admin existing doesn't necessarily mean it's wired up to give sudo rights — it might just be a plain group with no special permissions. Check what's in /etc/sudoers or /etc/sudoers.d/:

-> grep -r admin /etc/sudoers /etc/sudoers.d/ 2>/dev/null

```
root@nodejs-app:~# grep -r admin /etc/sudoers /etc/sudoers.d/ 2>/dev/null
/etc/sudoers:# Members of the admin group may gain root privileges
/etc/sudoers:%admin ALL=(ALL) ALL
root@nodejs-app:~#
```

that confirms admin is wired up for sudo on this server: /etc/sudoers has %admin ALL=(ALL) ALL, meaning any member of the admin group gets full root privileges via sudo. So your original playbook (group: admin) is correct and will give astrid sudo access once applied.

Now I need to refactor the play to deploy nodes app to use the new user ‘astrid’ instead of ‘root’

```yaml
- name: Deploy nodejs app
  hosts: 46.101.49.110
  tasks:
    - name: Unpack the nodejs file
      unarchive:
        src: /Users/astrid/PythonProject/Ansible/nodejs-app/nodejs-app-1.0.0.tgz
        dest: /root/
    - name: Install dependencies
      npm:
        path: /root/package
    - name: Start the application
      command:
        chdir: /root/package/app
        cmd: node server
      async: 1000
      poll: 0
    - name: Ensure app is running
      shell: ps aux | grep node
      register: app_status
    - debug: msg={{app_status.stdout_lines}}

```

In linux the nome directory for the new user ‘astrid’ is /home/astrid

```yaml
- name: Deploy nodejs app
  hosts: 46.101.49.110
  tasks:
    - name: Unpack the nodejs file
      unarchive:
        src: /Users/astrid/PythonProject/Ansible/nodejs-app/nodejs-app-1.0.0.tgz
        dest: /home/astrid
    - name: Install dependencies
      npm:
        path: /home/astrid/package
    - name: Start the application
      command:
        chdir: /home/astrid/package/app
        cmd: node server
      async: 1000
      poll: 0
    - name: Ensure app is running
      shell: ps aux | grep node
      register: app_status
    - debug: msg={{app_status.stdout_lines}}
```

Next, we need to tell ansible to use the new user ‘astrid’ and for that we use the attribute ‘become_user’ and enable that attribute with ‘become’ set to true as it is false by default

Privilege escalation: become

- 'become_user' = set to user with desired privileges.
  Default is *root*
- 'become' allows you to 'become' another user,
  different from the user that logged into the machine

So the play is now like this:

```
22  - name: Deploy nodejs app
23    hosts: 46.101.49.110
24    become: true
25    become_user: astrid
26    tasks:
27      - name: Unpack the nodejs file
28        unarchive:
```

Let's execute the playbook again
-> ansible-playbook -i hosts deploy-node.yaml

```
TASK [Create linux user] ********************************************************************************************************
changed: [46.101.49.110]

PLAY [Deploy nodejs app] ********************************************************************************************************

TASK [Gathering Facts] **********************************************************************************************************
[WARNING]: Module remote_tmp /home/astrid/.ansible/tmp did not exist and was created with a mode of 0700, this may cause issues
 when running as another user. To avoid this, create the remote_tmp dir with the correct permissions manually
ok: [46.101.49.110]

TASK [Unpack the nodejs file] ***************************************************************************************************
changed: [46.101.49.110]

TASK [Install dependencies] *****************************************************************************************************
changed: [46.101.49.110]
```

```
TASK [Start the application] ***************************************************************************************************
changed: [46.101.49.110]

TASK [Ensure app is running] ***************************************************************************************************
changed: [46.101.49.110]

TASK [debug] *******************************************************************************************************************
ok: [46.101.49.110] => {
    "msg": [
        "astrid     39317 51.8  6.2 618056 61360 ?        Rl   12:07   0:00 node server",
        "astrid     39335  0.0  0.1   2800  1920 pts/4    S+   12:07   0:00 /bin/sh -c ps aux | grep node",
        "astrid     39337  0.0  0.2   7076  2164 pts/4    S+   12:07   0:00 grep node"
    ]
}

PLAY RECAP *********************************************************************************************************************
46.101.49.110              : ok=11   changed=6    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0

~/Pyt/Ansible  Module_15/Ansible_chapter-11 !1 ?3                                                    ✓  50s  13:07:36
```

And we can see it is running using ‘astrid’ user and we can check the droplet as well

```
root@nodejs-app:~# pwd
/root
root@nodejs-app:~# ls
package
root@nodejs-app:~# cd /home/
root@nodejs-app:/home# ls
astrid
root@nodejs-app:/home# cd astrid/
root@nodejs-app:/home/astrid# ls
package
root@nodejs-app:/home/astrid# ps aux | grep node
astrid     39317  0.4  5.9 618312 58432 ?        Sl   12:07   0:00 node server
root       39358  0.0  0.2   7076  2164 pts/2    S+   12:10   0:00 grep --color=auto node
root@nodejs-app:/home/astrid#
```

We can with users per play using ‘become’ and ‘become_user’ attributes 💡